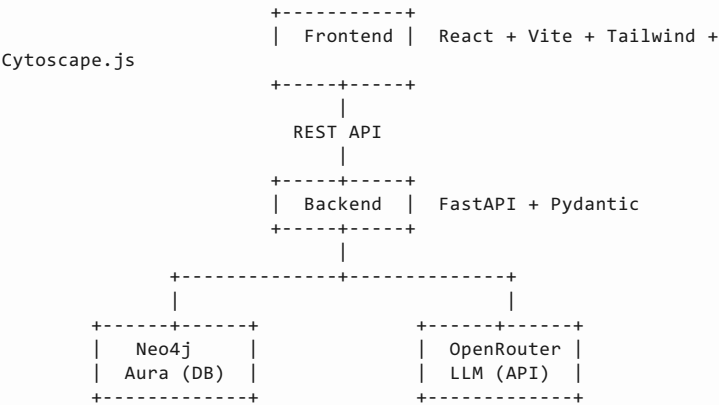


DODGE - Architecture

Graph-based data modeling and natural language query system for SAP Order-to-Cash data. Built with FastAPI, Neo4j (Aura), OpenRouter LLM, React, and Cytoscape.js.

System Overview



Two primary pipelines:

Data Pipeline: JSONL files -> Schema inference -> Relationship mapping -> Graph construction -> Neo4j

Query Pipeline: User question -> Guardrails -> Intent extraction (LLM) -> Template selection -> Cypher generation -> Validation -> Execution -> Formatted response

Backend (FastAPI)

Folder Structure

```

backend/
├── app/
│   ├── main.py # FastAPI app, CORS, lifespan,
router registration
│   └── config/
│       ├── settings.py # Pydantic BaseSettings (Neo4j,
│       └── constants.py # APP_NAME, BATCH_SIZE,
BLOCKED_CYPHER_KEYWORDS
│   └── ingestion/
│       ├── jsonl_loader.py # Reads entity folders,
concatenates JSONL partitions into DataFrames
│       └── schema_inference.py # ID column detection (suffix
heuristics + cardinality), candidate joins (Jaccard overlap)
│   └── modeling/
│       ├── graph_schema.py # NodeSchema, EdgeSchema,
GraphSchema (Pydantic models)
│       └── relationship_mapper.py # Builds GraphSchema from
DataFrames + joins, export/import/override config
│   └── graph/
│       ├── neo4j_client.py # Neo4j driver wrapper (execute,
execute_write, batch_execute, verify)
│       └── graph_builder.py # MERGE-based node/edge creation
with composite key support, batch inserts
│   └── llm/
│       ├── prompt_manager.py # Built-in + file-based prompt
templates with {placeholder} rendering
│       ├── intent_extractor.py # LLM call via OpenRouter,
returns QueryIntent (type, entities, filters, confidence)
│       ├── cypher_generator.py # Pure parameter filling into
${param} template placeholders (no LLM)
│       ├── template_registry.py # Loads .cypher files with
frontmatter, selects template by intent type
│       └── guardrails.py # Input (blocked patterns), domain
(entity whitelist), output (blocked Cypher keywords)
│   └── query/
│       ├── query_router.py # 8-step pipeline orchestration:
guardrail -> intent -> template -> cypher -> validate
│       ├── query_validator.py # Validates Cypher labels/types
against GraphSchema via regex extraction
│       ├── execution_engine.py # Runs Cypher via Neo4jClient,
measures time, delegates to PathExtractor
│       ├── path_extractor.py # Extracts GraphNode/GraphEdge
from raw Neo4j records for visualization
│       └── response_formatter.py # Builds QueryResponse: answer,
explanation, nodes, edges, metadata
│   └── services/
│       ├── graph_service.py # Ingestion orchestration: load -
> infer -> map -> build
│       └── query_service.py # Query orchestration: route ->
execute -> format
│   └── api/
│       ├── routes_graph.py # Graph/ingestion REST endpoints
│       └── routes_query.py # Natural language query REST
endpoint
│   └── utils/
│       ├── logger.py # Structured logging
(setup_logger, get_logger)
│       └── helpers.py # ensure_directory(),
sanitize_label() (snake_case -> PascalCase)
├── templates/
│   └── cypher/ # Cypher query templates with --
frontmatter
│       ├── find_node.cypher
│       ├── find_path.cypher
│       ├── list_neighbors.cypher
│       ├── aggregate.cypher
│       └── count.cypher
├── graph_schema_curated.json # Hand-curated schema with
correct composite keys and 24 clean edges
├── requirements.txt # fastapi, uvicorn, pydantic,
pydantic-settings, pandas, neo4j, openai, python-dotenv
├── .env # NEO4J_URI, NEO4J_USER,
NEO4J_PASSWORD, OPENROUTER_API_KEY, LLM_MODEL

```

API Endpoints

Graph Management (/api/v1/graph)

Method	Endpoint	Description
POST	/graph/preview	Load JSONL data + infer schema. Returns proposed GraphSchema for review. No data written to Neo4j.
POST	/graph/build	Accept a (possibly user-edited) GraphSchema, build graph in Neo4j. Supports clear_existing flag.
DELETE	/graph/clear	Delete all nodes and relationships from Neo4j.
GET	/graph/explore?limit=200	Fetch a sample of nodes and relationships for visualization.
GET	/graph/status	Connection status, node labels, relationship types, counts.

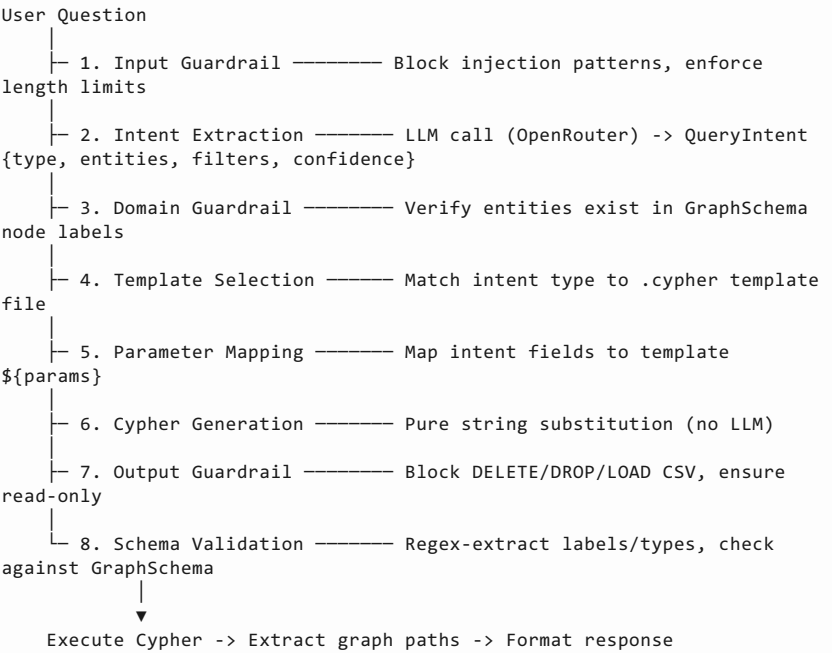
Query (/api/v1/query)

Method	Endpoint	Description
POST	/query/ask	Natural language question -> full pipeline -> structured response with answer, explanation, Cypher used, nodes, edges, metadata.

Health (/api/v1)

Method	Endpoint	Description
GET	/health	App health check.

Query Pipeline (8 Steps)

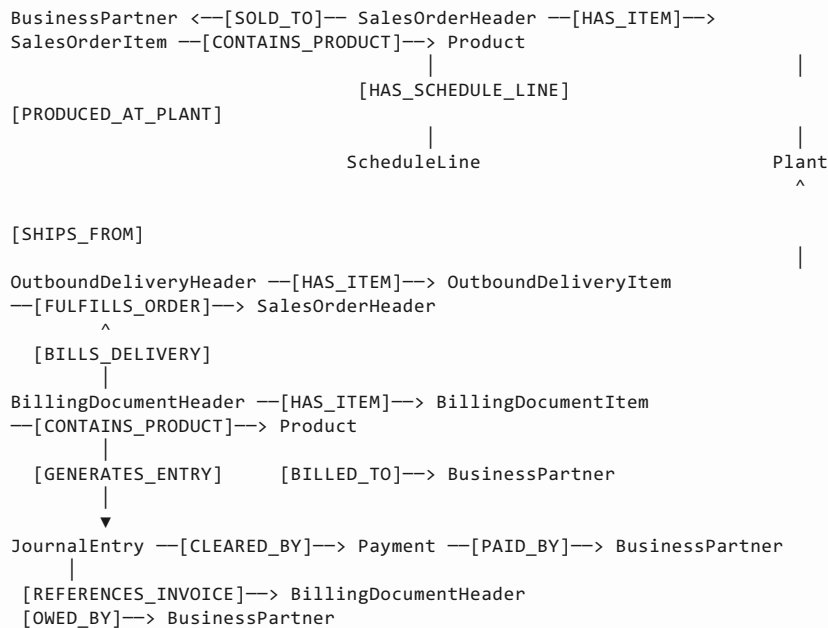


Key Design Principles

- **LLM= translation only.** The LLM extracts intent/entities. All Cypher is generated deterministically from templates.
- **Pydantic models co-located** in their owning modules. No separate `models/` package.
- **MERGE-based ingestion.** Idempotent node/edge creation using composite keys.
- **Two-step ingestion.** Preview (infer) -> user review -> build. Prevents data loss from auto-inference errors.
- **No business logic in routes.** Routes delegate to services, services delegate to domain modules.

Data Model (SAP Order-to-Cash)

19 entity types from SAP O2C dataset. 24 curated relationships following the business flow:



Master data: Product -[AVAILABLE_AT]-> ProductPlant -[LOCATED_AT]-> Plant, descriptions, addresses, customer assignments.

Frontend (React + TypeScript)

Folder Structure

```
frontend/src/
├── api/
│   └── client.ts           # Axios wrapper for all backend endpoints
├── components/
│   └── TopBar.tsx          # Navigation bar (Explorer / Ingestion
│                           tabs)
│   ├── CytoscapeGraph.tsx # Cytoscape.js graph canvas with color-
│                           coded nodes by label
│   └── ChatPanel.tsx       # Chat sidebar: NL query input, response
│                           display with Cypher + metadata
├── pages/
│   ├── Explorer.tsx        # Main view: status bar + graph canvas +
│                           chat panel
│   └── Ingestion.tsx       # Two-step ingestion: preview -> schema
│                           editor -> build
├── types/
│   └── index.ts            # TypeScript interfaces mirroring backend
Pydantic models
├── App.tsx                # BrowserRouter with route setup
├── main.tsx               # Entry point
└── index.css              # Tailwind CSS imports
```

Pages

Explorer (/)

- **Status bar** — Neo4j connection indicator, node/relationship/label counts
- **Graph canvas** — Cytoscape.js visualization. Auto-loads full graph on mount via /graph/explore. Color-coded nodes by label, directed edges with type labels, cose layout.
- **View toggle** — Switch between “Full Graph” (all data) and “Query Results” (from chat)
- **Chat panel** — Type natural language questions. Displays answer, explanation, generated Cypher, node/edge counts, execution time. Query results auto-render on the graph canvas.

Ingestion (/ingest)

- **Clear Database** — Red button to wipe Neo4j (DELETE /graph/clear). Required before re-ingesting on Aura free tier (400k relationship limit).
- **Preview Schema** — Loads JSONL data, runs inference, returns proposed schema. No writes.
- **Schema Editor** — JSON textarea showing inferred schema. User can edit composite keys, add/remove edges, fix relationship names before building.
- **Node/Edge summary cards** — Quick overview of detected labels and relationships.
- **Build Graph** — Sends (edited) schema to backend, builds graph in Neo4j. Shows build report with per-label node counts, per-type edge counts, and errors.

Tech Stack

Layer	Technology
Framework	React 19 + TypeScript
Build	Vite
Styling	Tailwind CSS
Graph Viz	Cytoscape.js
HTTP	Axios
Routing	React Router DOM

Dev server proxies /api to localhost:8000 (FastAPI backend).

Infrastructure

Component	Technology	Notes
Graph DB	Neo4j Aura (cloud)	Free tier: 200k nodes, 400k relationships
LLM	OpenRouter API	OpenAI-compatible. Model configurable via LLM_MODEL env var.
Backend	FastAPI + Uvicorn	Python 3.11+
Frontend	Vite dev server	Proxies API calls to backend

Environment Variables (.env)

```
NEO4J_URI=neo4j+s://<instance>.databases.neo4j.io
NEO4J_USER=neo4j
NEO4J_PASSWORD=...
OPENROUTER_API_KEY=sk-or-...
OPENROUTER_BASE_URL=https://openrouter.ai/api/v1
LLM_MODEL=<model-id>
LOG_LEVEL=INFO
```

Running

```
# Backend
cd backend
pip install -r requirements.txt
uvicorn app.main:app --reload
```

```
# Frontend
cd frontend
npm install
npm run dev
```

Backend: <http://localhost:8000> (Swagger UI at /docs) Frontend:
<http://localhost:5173>